



¿Cómo definir un lenguaje?

Teoría de la Computación
Marzo 2026

Material realizado a partir de las presentaciones de LpC



En caso de dudas, o querer profundizar...

- Sobre interpretes: <https://www.cse.chalmers.se/edu/year/2012/course/DAT150/lectures/plt-book.pdf> (Pags. 95-99)
- BNF: <https://condor.depaul.edu/ichu/csc447/notes/wk3/BNF.pdf>
- BNF: <https://cs61.seas.harvard.edu/site/2021/BNFGrammars/>
- BNF: https://ada.computerscience.org/concepts/trans_bnf (Secciones 2 y 3)



Repaso

Definición 1 (Lenguaje formal)

Un *lenguaje formal* consiste en:

- 1 **Alfabeto.** Un conjunto finito de símbolos que contiene los símbolos admisibles en el lenguaje.
- 2 **Gramática.** Un conjunto de reglas de sintaxis que describen como se forman las expresiones válidas del lenguaje a partir del alfabeto. Dichas expresiones son llamadas *fórmulas bien formadas* (o simplemente *fórmulas*).

- La gramática la vamos a especificar en (una variante más liviana de) la notación **BNF** (*Backus Naur Form*).
- Usualmente es conveniente identificar el lenguaje formal con el conjunto de fórmulas que su gramática describe.



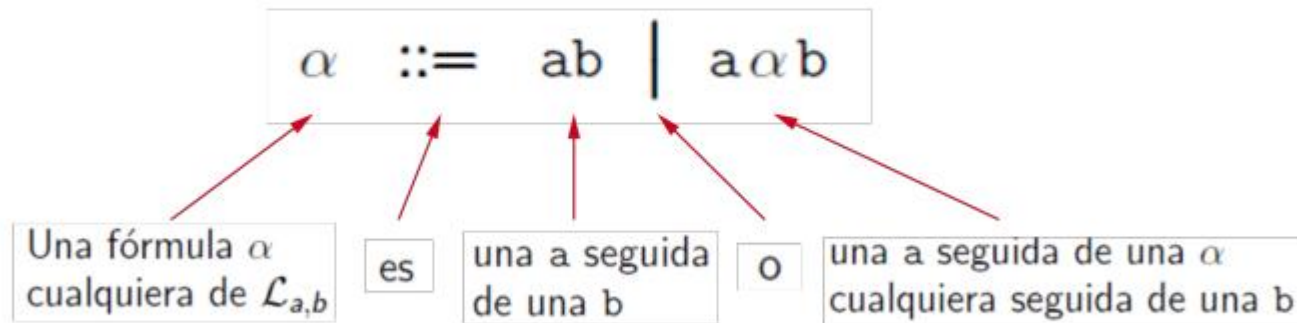
Ejemplo

Vamos a definir el lenguaje de a's seguidas de la misma cantidad de b's.

Presentamos un ejemplo simple para ilustrar ideas básicas.

Vamos a definir el lenguaje $\mathcal{L}_{a,b}$ de *a's seguidas de la misma cantidad de b's*.

- Alfabeto = $\{ a, b \}$
- Gramática en notación BNF:



Convención: usamos símbolos $\alpha, \beta, \gamma, \dots$ para denotar fórmulas arbitrarias del lenguaje, son meta-variables sintácticas.



Podemos pensar en $\mathcal{L}_{a,b}$ como el conjunto de fórmulas que su gramática describe o genera:

$$\mathcal{L}_{a,b} = \{ab, aabb, aaabbb, aaaabbbb, \dots\}$$

Importante: $\mathcal{L}_{a,b}$ es decidable, es decir que existe un algoritmo que puede determinar si una expresión está o no está en el lenguaje.

Ejemplo 3 (Expresiones que no están en el lenguaje)

- ❶ $a \notin \mathcal{L}_{a,b}$
- ❷ $abb \notin \mathcal{L}_{a,b}$
- ❸ $ac \notin \mathcal{L}_{a,b}$

- Las dos reglas de $\mathcal{L}_{a,b}$ las podemos expresar alternativamente de la siguiente manera:

$$R_1 \frac{}{ab \in \mathcal{L}_{a,b}} \qquad R_2 \frac{\alpha \in \mathcal{L}_{a,b}}{a\alpha b \in \mathcal{L}_{a,b}}$$

Si queremos verificar que aaabbb es una fórmula del lenguaje, podemos proceder aplicando las reglas “hacia atrás”:

$$? \frac{?}{aaabbb \in \mathcal{L}_{a,b}}$$

Las dos reglas de $\mathcal{L}_{a,b}$ las podemos expresar alternativamente de la siguiente manera:

$$R_1 \frac{}{ab \in \mathcal{L}_{a,b}} \qquad R_2 \frac{\alpha \in \mathcal{L}_{a,b}}{a\alpha b \in \mathcal{L}_{a,b}}$$

Si queremos verificar que $aaabbb$ es una fórmula del lenguaje, podemos proceder aplicando las reglas “hacia atrás”:

$$\begin{array}{c} R_1 \frac{}{ab \in \mathcal{L}_{a,b}} \\ R_2 \frac{}{aabb \in \mathcal{L}_{a,b}} \\ R_2 \frac{}{aaabbb \in \mathcal{L}_{a,b}} \end{array}$$



Ejemplo LP:

Definición 2 ($\mathcal{L}_{LP}$ en BNF)

$$\alpha, \beta ::= p_i \mid (\neg \alpha) \mid (\alpha \wedge \beta) \mid (\alpha \vee \beta) \mid (\alpha \supset \beta) \mid (\alpha \leftrightarrow \beta)$$

donde $p_i \in Var$.

Árbol de sintaxis

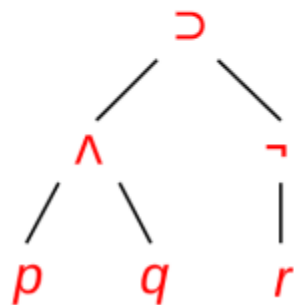
Hay una correspondencia uno a uno entre cada fórmula de nuestro lenguaje y un árbol de sintaxis. No hay ambigüedad.

Sintaxis lineal

$((p \wedge q) \supset (\neg r)) \in \mathcal{L}_{LP}$

↑
Conectiva principal
(la de “más afuera”)

Árbol de sintaxis



Raíz

= Conectiva principal

Nodos internos

= Conectivas

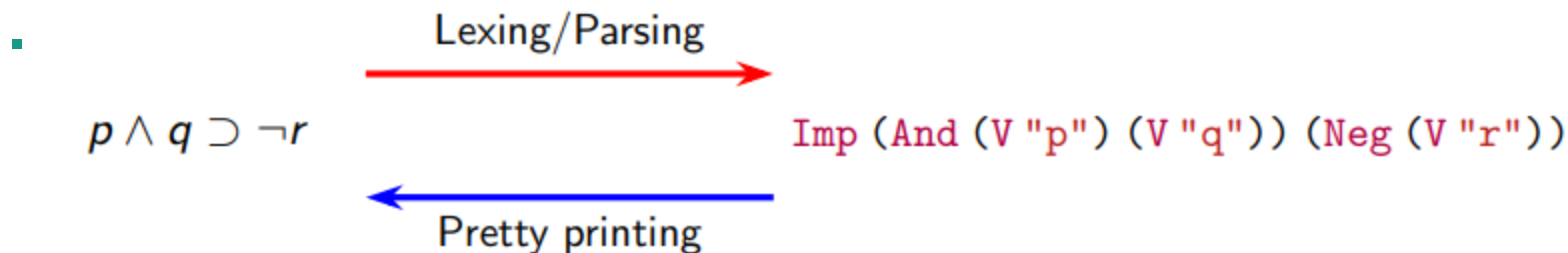
Hojas

= Variables

	$\mathcal{L}_{LP}$	Haskell
Alfabeto	$Var \cup \{\neg, \wedge, \vee, \supset, \leftrightarrow, (,)\}$	<code>type Var = String</code>
Gramática	$\alpha, \beta ::= p_i \quad , p_i \in Var$ $\quad (\neg \alpha)$ $\quad (\alpha \wedge \beta)$ $\quad (\alpha \vee \beta)$ $\quad (\alpha \supset \beta)$ $\quad (\alpha \leftrightarrow \beta)$	<code>data L = V Var</code> $\quad \text{Neg } L$ $\quad \text{And } L \ L$ $\quad \text{Or } L \ L$ $\quad \text{Imp } L \ L$ $\quad \text{Iff } L \ L$

Sintaxis concreta

Sintaxis abstracta



En Haskell, operadores binarios prefijos se pueden expresar localmente infijos usando *backticks*:

```
((V "p") `And` (V "q")) `Imp` (Neg (V "r"))
```



Ejemplo: Evaluador de expresiones aritméticas



¿Qué lenguajes tengo que definir?

- Metalenguaje -> Backus Naur Form (BNF), Deducción Natural (DN)
- Lenguaje objeto -> Expresiones aritméticas
- Lenguaje de implementación -> Haskell



Sintaxis (abstracta) en BNF

Quiero poder tener:

- Literales numéricos
- Suma y resta de dos expresiones
- El opuesto de una expresión

Ejemplos de expresiones...



Semántica: especificando el intérprete

- Utilizaremos reglas de inferencia
- El sistema de reglas se llama **semántica operacional**
- Especificamos las reglas para saber cómo evaluar las expresiones y ejecutar las sentencias o los programas
- El juicio más básico para evaluar expresiones es: $e \Downarrow v$

Que se lee como: la expresión e evalúa al valor v en el ambiente/entorno γ

¿Qué es un valor?



Pasarlo a Haskell

- Definir estructuras a utilizar
- Programar la función evaluación



Variante 1:

Ahora quiero poder tener una memoria ya cargada y consultar variables



Variante 2

Además de lo anterior, quiero poder realizar asignaciones ($x := e$)

Dato: esta variante pero de expBool está en el repo:

https://github.com/diegoacu1234/TC_Marzo26/blob/main/S02_Practico0/expBool.hs